République tunisienne
Ministère de l'Enseignement supérieur et de la Recherche scientifique
TEK-UP University
École privée supérieure de technologie et d'ingénierie

Conception et Implémentation d'un Système Intelligent de Gestion d'Atelier Automobile

Présenté en vue de l'obtention de la

Cycle d'Ingénieur en Informatique

Parcours : Génie Logiciel

Réalisé par

Ihebeddine Saafi | Arbi Hazbri | Louay Lewati

MecaManage

Thème du projet

Année Universitaire : 2025–2026

Résumé

MecaRage est une plateforme logicielle complète dédiée à la gestion moderne et intelligente des ateliers mécaniques. Elle repose sur une architecture distribuée à plusieurs couches : un back-end **ASP.NET Core 9** en Clean Architecture (CQRS et MediatR), un front-end **Angular 21** avec Tailwind CSS, un service d'intelligence artificielle fondé sur **FastAPI** et **Ollama** (LLM local) pour le diagnostic contextuel par RAG (Retrieval-Augmented Generation), et un module mobile **Android** (Java) intégrant **Firebase** pour les notifications en temps réel.

La plateforme couvre l'intégralité du cycle de vie d'une intervention mécanique : dépôt des symptômes, diagnostic IA, prise de rendez-vous, affectation des mécaniciens, rapport d'examen, facturation, approbation client, réparation, contrôle qualité et paiement — le tout tracé par un suivi de cycle de vie unifié (*Intervention Lifecycle Tracker*).

Mots-clés : Gestion d'atelier, Clean Architecture, CQRS, Angular 21, ASP.NET Core 9, Firebase, Intelligence Artificielle, RAG, Android, MySQL.

Abstract

MecaRage is a comprehensive software platform for the modern and intelligent management of mechanical workshops. It is built on a multi-layer distributed architecture : an **ASP.NET Core 9** back-end with Clean Architecture (CQRS and MediatR), an **Angular 21** front-end with Tailwind CSS, an AI service based on **FastAPI** and **Ollama** (local LLM) for RAG-based contextual diagnosis, and an **Android** mobile module (Java) integrating **Firebase** for real-time notifications.

The platform covers the complete lifecycle of a mechanical intervention : symptom submission, AI diagnosis, appointment scheduling, mechanic assignment, examination report, invoicing, client approval, repair, quality validation and payment — all tracked by a unified Intervention Lifecycle Tracker.

Keywords : Workshop management, Clean Architecture, CQRS, Angular 21, ASP.NET Core 9, Firebase, Artificial Intelligence, RAG, Android, MySQL.

Table des matières

Résumé **i**

Abstract **ii**

Table des matières **iv**

Table des figures **v**

Liste des tableaux **vi**

1 Les Bases **1**

1.1 Introduction 1

1.2 Présentation de l'entreprise 1

1.3 Présentation du projet 1

1.3.1 Problématique 1

1.3.2 Étude de l'existant 2

1.3.3 Solution proposée 2

1.4 Conclusion 3

2 Spécification Fonctionnelle et Technique **4**

2.1 Introduction 4

2.2 Spécification Fonctionnelle 4

2.2.1 Exigences Fonctionnelles 4

2.2.2 Diagramme de cas d'utilisation global 5

2.2.3 Diagramme de classes global 5

2.2.4 Backlog Produit 5

2.3 Spécification Technique 9

2.3.1 Exigences Non-Fonctionnelles 9

2.3.2 Outils et Environnement de Développement 10

2.4 Conclusion 10

3 Sprint 1 : Authentification et Gestion des Profils **11**

3.1 Introduction 11

3.2 Identification du Backlog Sprint 1 11

3.3 Conception 12

3.3.1 Diagramme de cas d'utilisation — Sprint 1 12

3.3.2 Diagramme de classes — Sprint 1 13

3.3.3 Diagramme de séquence 13
3.4 Conclusion 13

4 Sprint 2 : Gestion des Données de Référence 16
4.1 Introduction 16
4.2 Identification du Backlog Sprint 2 16
4.3 Conception 17
4.3.1 Diagramme de cas d'utilisation — Sprint 2 17
4.3.2 Diagramme de classes — Sprint 2 18
4.3.3 Diagramme de séquence 18
4.4 Conclusion 18

5 Sprint 3 : Rapports de Symptômes, Diagnostic IA et Rendez-vous 20
5.1 Introduction 20
5.2 Identification du Backlog Sprint 3 20
5.3 Conception 21
5.3.1 Diagramme de cas d'utilisation — Sprint 3 21
5.3.2 Pipeline du Diagnostic IA 23
5.3.3 Diagramme de classes — Sprint 3 24
5.3.4 Diagramme de séquence 24
5.4 Conclusion 24

6 Sprint 4 : Opérations d'Atelier et Cycle de Vie des Interventions 27
6.1 Introduction 27
6.2 Identification du Backlog Sprint 4 28
6.3 Conception 29
6.3.1 Diagramme de cas d'utilisation — Sprint 4 29
6.3.2 Diagramme de classes — Sprint 4 31
6.3.3 Diagramme de séquence 31
6.4 Conclusion 31

Conclusion Générale 34

Bibliographie 36

Table des figures

1.1 Architecture globale de la plateforme MecaRage 2

2.1 Diagramme de cas d'utilisation global — MecaRage 6
2.2 Diagramme de classes global — Entités du domaine MecaRage 7

3.1 Diagramme de cas d'utilisation — Sprint 1 12
3.2 Diagramme de classes — Sprint 1 (Authentification) 13
3.3 Diagramme de séquence SC1 — Connexion (Login) 14

4.1 Diagramme de cas d'utilisation — Sprint 2 17
4.2 Diagramme de classes — Sprint 2 (Données de Référence) 18
4.3 Diagramme de séquence SC3 — Création d'un Garage 19

5.1 Diagramme de cas d'utilisation — Sprint 3 21
5.2 Pipeline RAG — Service de diagnostic IA 23
5.3 Diagramme de classes — Sprint 3 (Symptômes, IA et Rendez-vous) . . 24
5.4 Diagramme de séquence SC5 — Soumission de Symptômes et Diagnostic IA 25

6.1 Diagramme de cas d'utilisation — Sprint 4 29
6.2 Machine d'états — Intervention Lifecycle Tracker 30
6.3 Diagramme de classes — Sprint 4 (Atelier et Cycle de Vie) 31
6.4 Diagramme de séquence SC6 — Finalisation et Approbation du Devis . 32

Liste des tableaux

1.1 Problèmes identifiés dans le système actuel 2
1.2 Outils existants et leurs limites 2

2.1 Exigences fonctionnelles par rôle 4
2.2 Backlog Produit complet — MecaRage 8
2.3 Exigences non-fonctionnelles 9
2.4 Stack technologique de MecaRage 10

3.1 Backlog Sprint 1 — Authentification 11

4.1 Backlog Sprint 2 — Données de Référence 16

5.1 Backlog Sprint 3 — Symptômes, IA et Rendez-vous 20

6.1 Backlog Sprint 4 — Opérations d'Atelier 28
6.2 Récapitulatif des réalisations par sprint 34

Chapitre 1

Les Bases

*Ce chapitre présente le contexte général du projet **MecaRage** : la structure de l'entreprise commanditaire, la problématique à l'origine du projet et la solution proposée.*

1.1 Introduction

Dans un contexte de digitalisation croissante des métiers de la maintenance automobile, les ateliers mécaniques ressentent une pression de plus en plus forte pour moderniser leurs processus. MecaRage a été conçu pour répondre à ce besoin en offrant une solution numérique complète, couvrant l'ensemble de la chaîne opérationnelle d'un réseau d'ateliers multi-sites.

1.2 Présentation de l'entreprise

L'entreprise partenaire coordonne plusieurs *garages* répartis géographiquement, chacun disposant d'une équipe de mécaniciens supervisée par un chef d'atelier. La direction centrale assure la supervision globale des opérations et de la facturation.

1.3 Présentation du projet

1.3.1 Problématique

La gestion d'un réseau d'ateliers mécaniques implique une coordination complexe entre plusieurs parties prenantes : les clients qui déposent leurs véhicules, les mécaniciens qui exécutent les travaux, le chef d'atelier qui orchestre les ressources, et la direction qui supervise les finances.

Table 1.1. Problèmes identifiés dans le système actuel

Domaine	Problème	Conséquence
Communication	RDV par téléphone/WhatsApp	Pas de traçabilité
Rapports	Fiches papier mécanicien	Risque de perte, illisibilité
Facturation	Tableur Excel par garage	Pas de vision consolidée
Suivi	Aucun outil de statut en temps réel	Manque de visibilité client
Diagnostic	Réception sans pré-analyse	Perte de temps à l'accueil
Stock	Inventaire manuel pièces	Ruptures non anticipées

1.3.2 Étude de l'existant

L'existant se compose d'une combinaison d'outils non intégrés :

Table 1.2. Outils existants et leurs limites

Outil	Usage actuel	Limite principale
WhatsApp / SMS	Prise de RDV, communications	Informel, non traçable
Excel	Inventaire pièces, facturation	Pas de collaboration temps réel
Papier	Rapports mécaniciens	Perte, archivage difficile
Tableau blanc	Suivi visuel de l'atelier	Non numérique, local uniquement

1.3.3 Solution proposée

MecaRage est une plateforme numérique unifiée couvrant l'intégralité du cycle de vie d'une intervention mécanique.

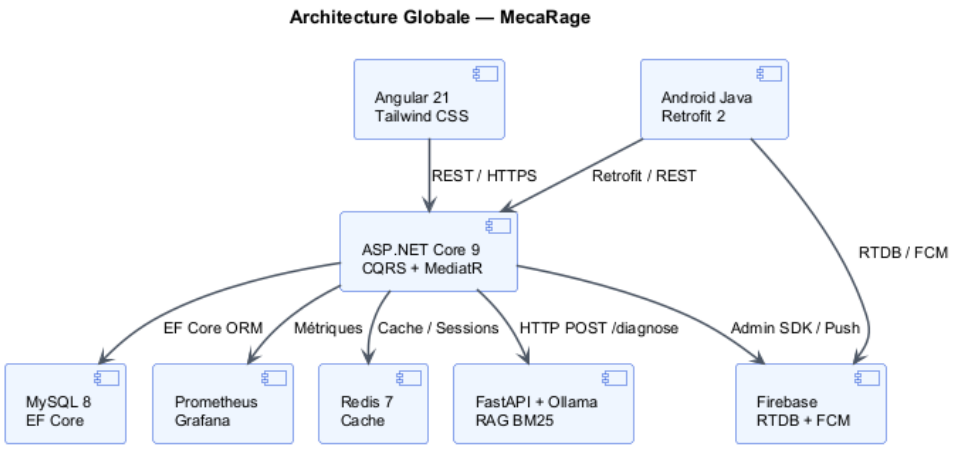


Figure 1.1. Architecture globale de la plateforme MecaRage

Les apports clés :

— **Application web** accessible par tous les rôles (Angular 21).
— **Back-end robuste** (ASP.NET Core 9, Clean Architecture, CQRS).
— **Service IA** (FastAPI + Google Gemini) pour le diagnostic préliminaire.
— **Module mobile Android** (Java) avec Firebase temps réel.
— **Gestion multi-tenant** : plusieurs entreprises sur la même plateforme.
— **Intervention Lifecycle Tracker** : suivi automatique sur 10 états.
— **Génération PDF** des factures via QuestPDF.

1.4 Conclusion

Ce chapitre a présenté le contexte métier qui a motivé la création de MecaRage. Le chapitre suivant détaille les spécifications fonctionnelles et techniques.

Chapitre 2

Spécification Fonctionnelle et Technique

Ce chapitre formalise les exigences fonctionnelles et non-fonctionnelles de la plateforme à travers le diagramme de cas d'utilisation global, le diagramme de classes du domaine, le backlog produit complet et les choix techniques.

2.1 Introduction

La phase de spécification garantit que l'équipe de développement et les parties prenantes partagent une vision commune des fonctionnalités à réaliser. Ce chapitre présente les outils de modélisation utilisés pour formaliser cette vision.

2.2 Spécification Fonctionnelle

2.2.1 Exigences Fonctionnelles

Table 2.1. Exigences fonctionnelles par rôle

N°	Acteur	Exigence
EF01	Tous	S'authentifier avec email et mot de passe (JWT)
EF02	Tous	Rafraîchir automatiquement le token d'accès
EF03	Tous	Modifier son mot de passe
EF04	SuperAdmin	Créer et gérer les tenants (entreprises)
EF05	AdminEntreprise	Créer et gérer les garages du tenant
EF06	AdminEntreprise	Créer et gérer le personnel (chefs, mécaniciens)
EF07	AdminEntreprise	Consulter les indicateurs de performance (KPI)

N°	Acteur	Exigence
EF08	AdminEntreprise	Enregistrer le paiement d'une intervention
EF09	Client	S'inscrire sur la plateforme
EF10	Client	Gérer ses véhicules (ajout, consultation)
EF11	Client	Soumettre un rapport de symptômes textuels
EF12	Système	Générer un diagnostic IA (Ollama RAG) automatiquement
EF13	Client	Réserver un rendez-vous dans la plage proposée
EF14	Client	Approuver ou refuser un devis
EF15	Client	Suivre le cycle de vie de son intervention
EF16	Client	Télécharger sa facture en PDF
EF17	ChefAtelier	Réviser les rapports et proposer une plage RDV
EF18	ChefAtelier	Approuver ou refuser des rendez-vous
EF19	ChefAtelier	Assigner des mécaniciens à une tâche de réparation
EF20	ChefAtelier	Réviser le rapport d'examen du mécanicien
EF21	ChefAtelier	Créer et finaliser une facture (devis)
EF22	ChefAtelier	Valider la réparation et notifier le client
EF23	Mécanicien	Consulter ses tâches assignées
EF24	Mécanicien	Soumettre un rapport d'examen (pièces, observations)
EF25	Mécanicien	Soumettre un rapport de fin de réparation
EF26	Système	Envoyer notifications in-app (MySQL) et push (FCM)
EF27	AdminEntreprise	Gérer l'inventaire des pièces détachées
EF28	Tous	Accéder via application Android (client et mécanicien)

2.2.2 Diagramme de cas d'utilisation global

2.2.3 Diagramme de classes global

2.2.4 Backlog Produit

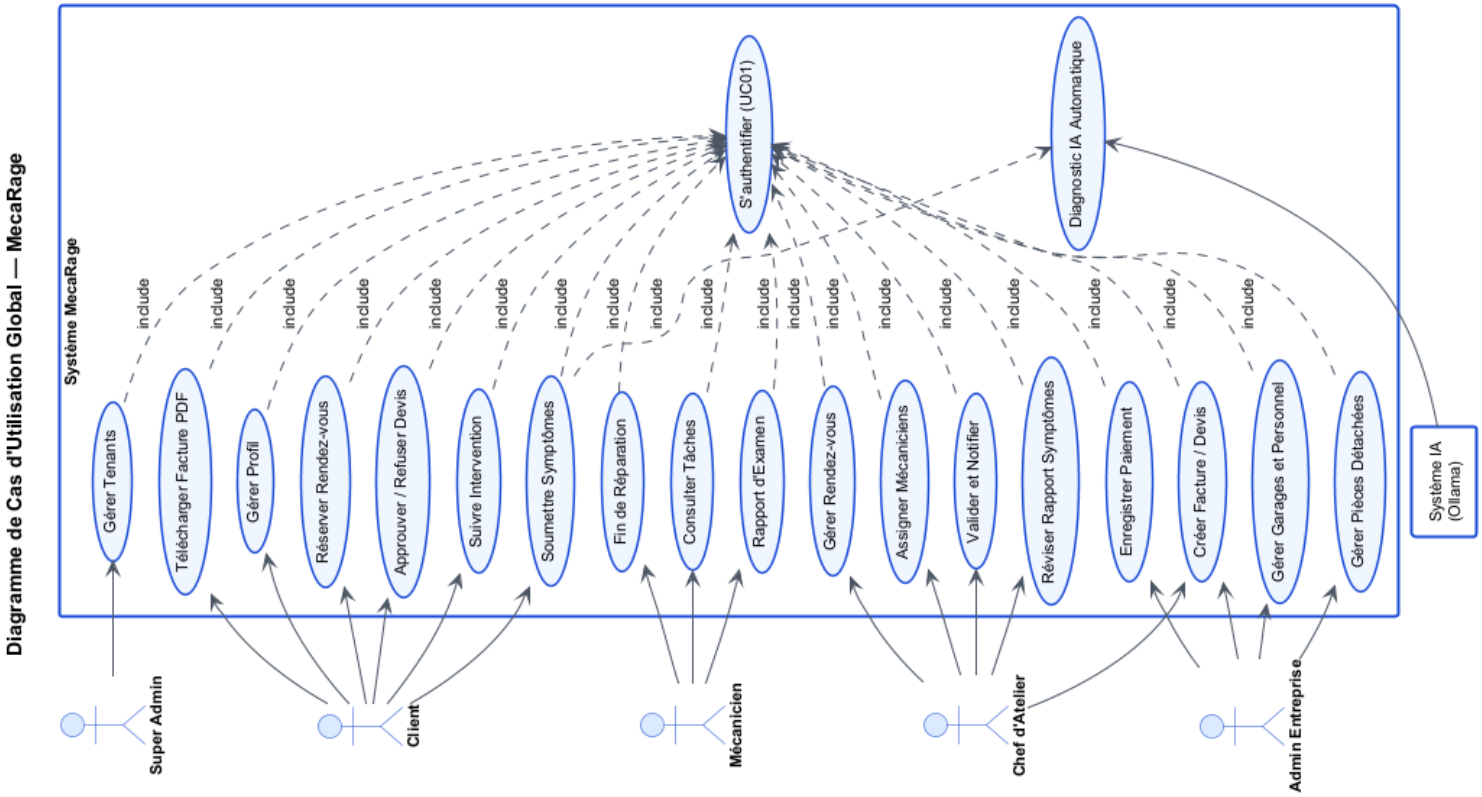


Figure 2.1. Diagramme de cas d'utilisation global — MecaRage

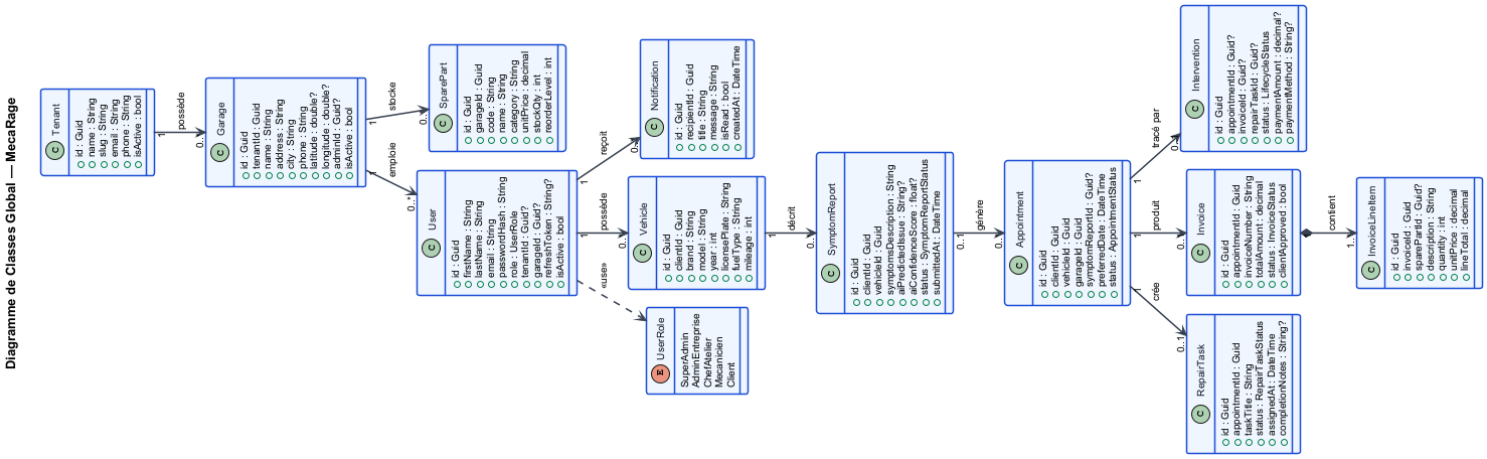


Figure 2.2. Diagramme de classes global — Entités du domaine MecaRage

Table 2.2. Backlog Produit complet — MecaRage

ID	En tant que	Je veux	Priorité	SP	Sprint
US01	Utilisateur	M'inscrire avec email et mot de passe	**Haute**	3	Sprint 1
US02	Utilisateur	Me connecter et recevoir un JWT	**Haute**	3	Sprint 1
US03	Utilisateur	Rafraîchir mon token automatiquement	**Haute**	2	Sprint 1
US04	Utilisateur	Modifier mon mot de passe	**Moyenne**	2	Sprint 1
US05	SuperAdmin	Créer et gérer les tenants	**Haute**	3	Sprint 2
US06	AdminEntreprise	Créer et gérer des garages	**Haute**	5	Sprint 2
US07	AdminEntreprise	Gérer le personnel	**Haute**	5	Sprint 2
US08	Client	Enregistrer et gérer ses véhicules	**Haute**	3	Sprint 2
US09	AdminEntreprise	Gérer l'inventaire des pièces	**Moyenne**	5	Sprint 2
US10	Client	Soumettre un rapport de symptômes	**Haute**	5	Sprint 3
US11	Système	Générer un diagnostic IA (RAG)	**Haute**	8	Sprint 3
US12	ChefAtelier	Réviser les rapports et proposer une plage RDV	**Haute**	5	Sprint 3
US13	Client	Réserver un rendez-vous	**Haute**	3	Sprint 3
US14	ChefAtelier	Approuver ou refuser un rendez-vous	**Haute**	3	Sprint 3
US15	ChefAtelier	Assigner des mécaniciens à une tâche	**Haute**	5	Sprint 4
US16	Mécanicien	Consulter ses tâches assignées	**Haute**	3	Sprint 4
US17	Mécanicien	Soumettre un rapport d'examen	**Haute**	5	Sprint 4
US18	ChefAtelier	Réviser et valider le rapport d'examen	**Haute**	3	Sprint 4

ID	En tant que	Je veux	Priorité	SP	Sprint
US19	ChefAtelier	Créer et finaliser une facture	**Haute**	5	Sprint 4
US20	Client	Approuver ou refuser un devis	**Haute**	3	Sprint 4
US21	Mécanicien	Soumettre la fin de réparation	**Haute**	3	Sprint 4
US22	ChefAtelier	Valider et notifier le client	**Haute**	3	Sprint 4
US23	AdminEntreprise	Enregistrer le paiement	**Haute**	3	Sprint 4
US24	Système	Tracer le cycle de vie automatiquement	**Haute**	8	Sprint 4
US25	Utilisateurs	Recevoir notifications in-app et push	**Moyenne**	5	Sprint 4
US26	Client	Télécharger la facture en PDF	**Moyenne**	3	Sprint 4
US27	Client/Mécanicien	Utiliser l'application Android native	**Moyenne**	13	Sprint 4
			Total Story Points	**104 SP**	

2.3 Spécification Technique

2.3.1 Exigences Non-Fonctionnelles

Table 2.3. Exigences non-fonctionnelles

Critère	Exigence
Performance	Temps de réponse API < 300 ms pour 95% des requêtes
Sécurité	JWT HS256, hachage bcrypt, isolation multi-tenant, CORS contrôlé
Scalabilité	Architecture stateless ; front-end découplé ; services indépendants
Disponibilité	Docker Compose avec healthchecks et `restart: unless-stopped`
Maintenabilité	Clean Architecture, CQRS, tests xUnit, couverture $\geq 70\%$
Compatibilité	Android 8+ (API 26) ; navigateurs modernes (Chrome, Firefox, Edge)

2.3.2 Outils et Environnement de Développement

Table 2.4. Stack technologique de MecaRage

Couche	Technologie	Version / Rôle
Front-end Web	Angular + Tailwind CSS	v21.1 / SPA réactive, dark mode
Back-end API	ASP.NET Core + MediatR	v9.0 / API REST, CQRS, Clean Arch.
Base de données	MySQL + Pomelo EF Core	8.0 / ORM relationnel, migrations
Cache	Redis	7.0 / sessions, invalidation cache
Service IA	Python FastAPI + Ollama	3.13 / RAG BM25 + LLM local
Mobile	Android Java + Firebase	SDK 26+ / RTDB + FCM
PDF	QuestPDF	2026.x / génération factures
Déploiement	Docker + Docker Compose	25+ / containerisation totale
Monitoring	Prometheus + Grafana	— / métriques et alertes
Gestion projet	Méthode Agile Scrum	4 sprints, 2 semaines/sprint

2.4 Conclusion

Ce chapitre a formalisé les besoins de la plateforme par 27 user stories organisées en 4 sprints (104 story points au total). Les chapitres suivants détaillent la conception et le développement de chaque sprint.

Chapitre 3

Sprint 1 : Authentification et Gestion des Profils

Ce premier sprint pose les fondations de la plateforme en implémentant le système d'authentification sécurisé par JWT et la gestion des profils. Il constitue le prérequis de toutes les fonctionnalités ultérieures.

3.1 Introduction

L'authentification est le socle de toute plateforme multi-utilisateurs. Elle garantit que chaque acteur accède uniquement aux ressources qui lui sont attribuées selon son rôle. Ce sprint implémente un système JWT stateless, adapté au déploiement multi-tenant et à l'architecture découplée front-end/back-end.

3.2 Identification du Backlog Sprint 1

Table 3.1. Backlog Sprint 1 — Authentification

ID	User Story	Priorité	SP
US01	En tant qu'utilisateur, je veux m'inscrire avec email et mot de passe	**Haute**	3
US02	En tant qu'utilisateur, je veux me connecter et recevoir un JWT	**Haute**	3
US03	En tant qu'utilisateur, je veux que mon token se rafraîchisse automatiquement	**Haute**	2
US04	En tant qu'utilisateur, je veux modifier mon mot de passe	**Moyenne**	2
		Total Sprint 1	**10 SP**

Tâches techniques :

1. Modèles de domaine `User` et `Tenant`.

2. Commandes CQRS : `RegisterCommand`, `LoginCommand`, `RefreshTokenCommand`, `ChangePasswordCommand`.
3. Middleware JWT et filtre global multi-tenant EF Core.
4. Endpoints REST dans `AuthController`.
5. Écrans Angular : `LoginComponent`, `SignupComponent`.
6. Intercepteur HTTP pour injection automatique du Bearer Token.

3.3 Conception

3.3.1 Diagramme de cas d'utilisation — Sprint 1

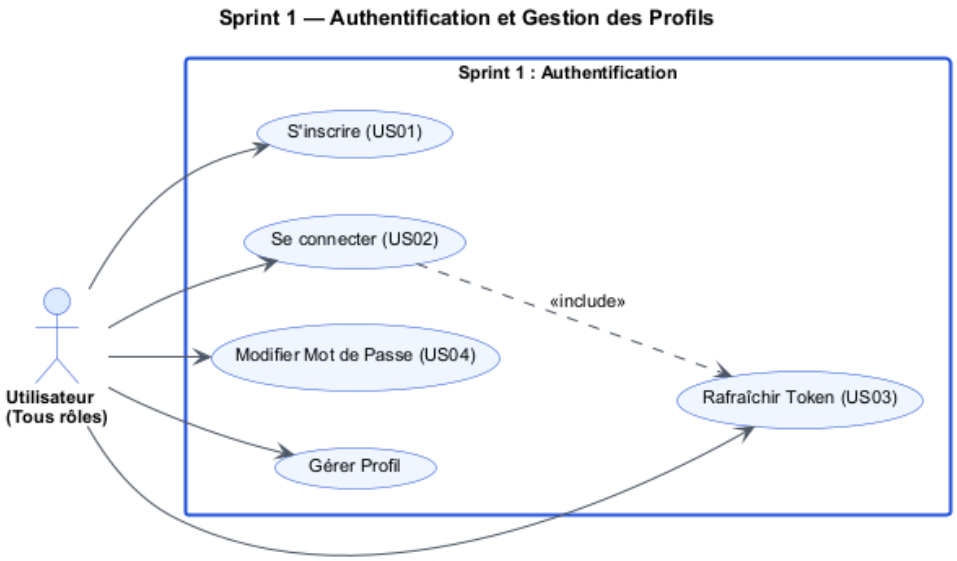


Figure 3.1. Diagramme de cas d'utilisation — Sprint 1

3.3.2 Diagramme de classes — Sprint 1

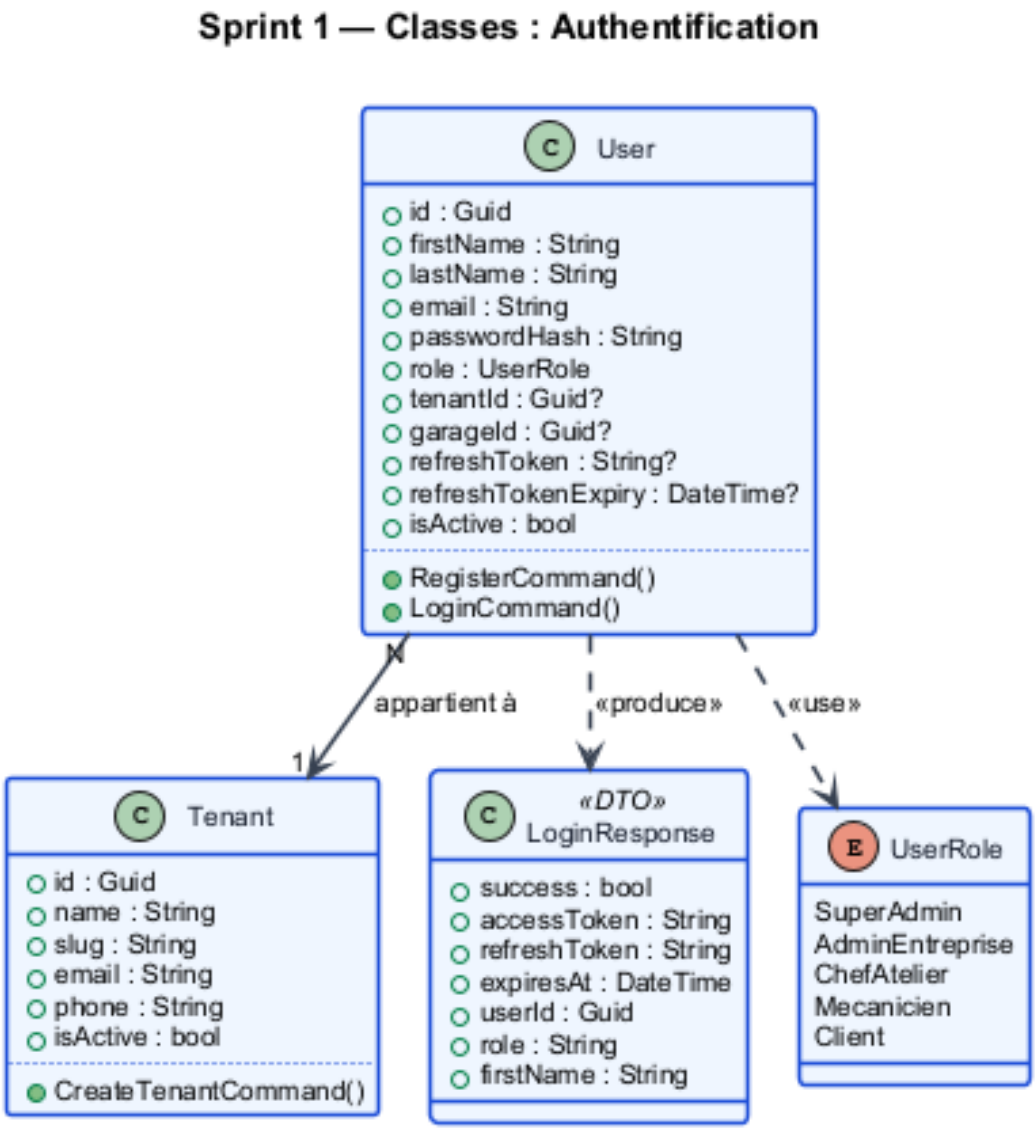


Figure 3.2. Diagramme de classes — Sprint 1 (Authentification)

3.3.3 Diagramme de séquence

SC1 — Connexion (US02) : fonctionnalité principale

L'utilisateur saisit ses identifiants. Le système valide, génère un JWT et un refresh token, les stocke en base, et les retourne au client.

3.4 Conclusion

Le Sprint 1 a établi l'infrastructure d'authentification complète de MecaRage. Le système JWT est opérationnel avec gestion du refresh token, isolation multi-tenant par filtre global EF Core (`HasQueryFilter`), et hachage BCrypt des mots de passe. Les

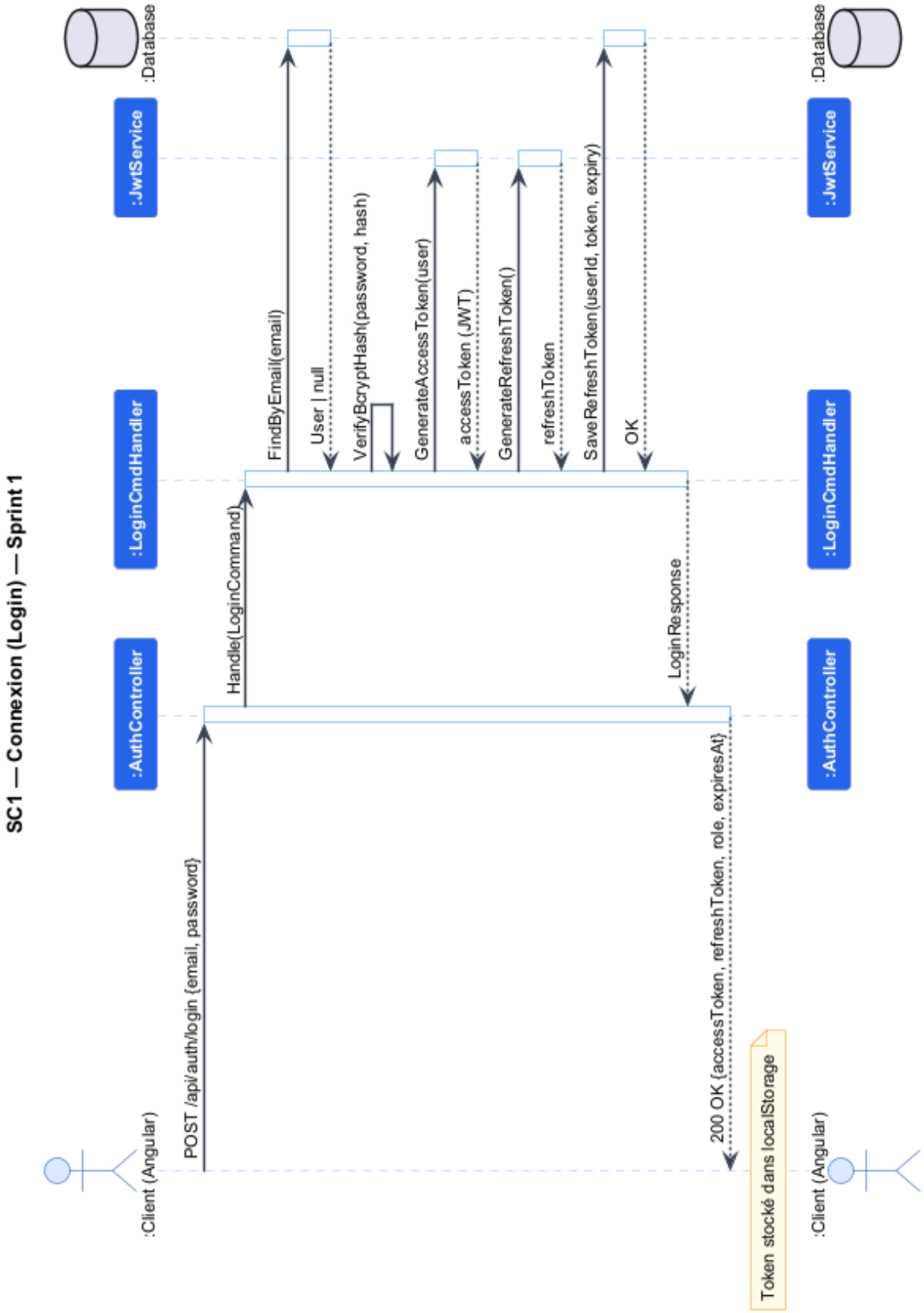


Figure 3.3. Diagramme de séquence SC1 — Connexion (Login)

interfaces Angular de connexion et d'inscription sont déployées avec support du mode sombre (dark mode Tailwind CSS).

Le Sprint 1 a établi l'infrastructure d'authentification complète de MecaRage. Le système JWT est opérationnel avec gestion du refresh token, isolation multi-tenant par filtre global EF Core (`HasQueryFilter`), et hachage BCrypt des mots de passe. Les interfaces Angular de connexion et d'inscription sont déployées avec support du mode sombre (dark mode Tailwind CSS).

Chapitre 4

Sprint 2 : Gestion des Données de Référence

Ce sprint met en place les données fondamentales : tenants, garages, personnel, véhicules et inventaire des pièces détachées.

4.1 Introduction

Les données de référence constituent le socle métier sans lequel aucune opération n'est possible : un garage doit exister pour accueillir des rendez-vous, un véhicule doit être enregistré pour soumettre des symptômes, et les pièces détachées doivent être référencées pour être incluses dans les factures.

4.2 Identification du Backlog Sprint 2

Table 4.1. Backlog Sprint 2 — Données de Référence

ID	User Story	Priorité	SP
US05	SuperAdmin : créer et gérer les tenants	**Haute**	3
US06	AdminEntreprise : créer et gérer des garages	**Haute**	5
US07	AdminEntreprise : gérer le personnel (chefs, mécaniciens)	**Haute**	5
US08	Client : enregistrer ses véhicules	**Haute**	3
US09	AdminEntreprise : gérer l'inventaire des pièces détachées	**Moyenne**	5
		Total Sprint 2	**21 SP**

Tâches techniques :

1. Entités de domaine : `Garage`, `Vehicle`, `SparePart`.

2. Commandes CQRS : `CreateGarageCommand`, `CreateVehicleCommand`, `CreateSparePartCommar`
   `CreateStaffCommand`.
3. Migrations EF Core pour les nouvelles tables.
4. Contrôleurs REST : `GaragesController`, `VehiclesController`, `SpareParsController`.
5. Politiques d'autorisation par rôle (`[Authorize(Roles = ...)]`).
6. Tableau de bord administrateur Angular.

4.3 Conception

4.3.1 Diagramme de cas d'utilisation — Sprint 2

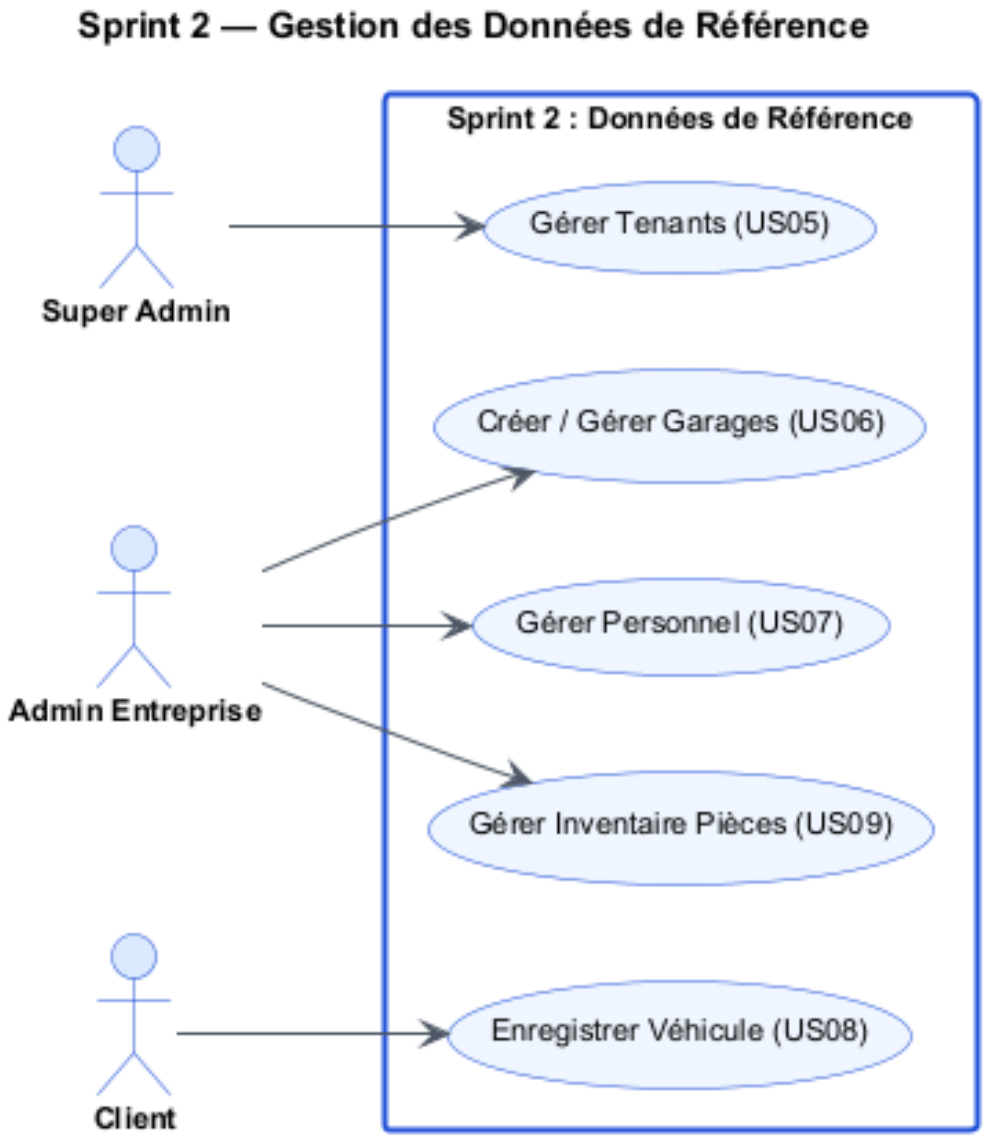


Figure 4.1. Diagramme de cas d'utilisation — Sprint 2

4.3.2 Diagramme de classes — Sprint 2

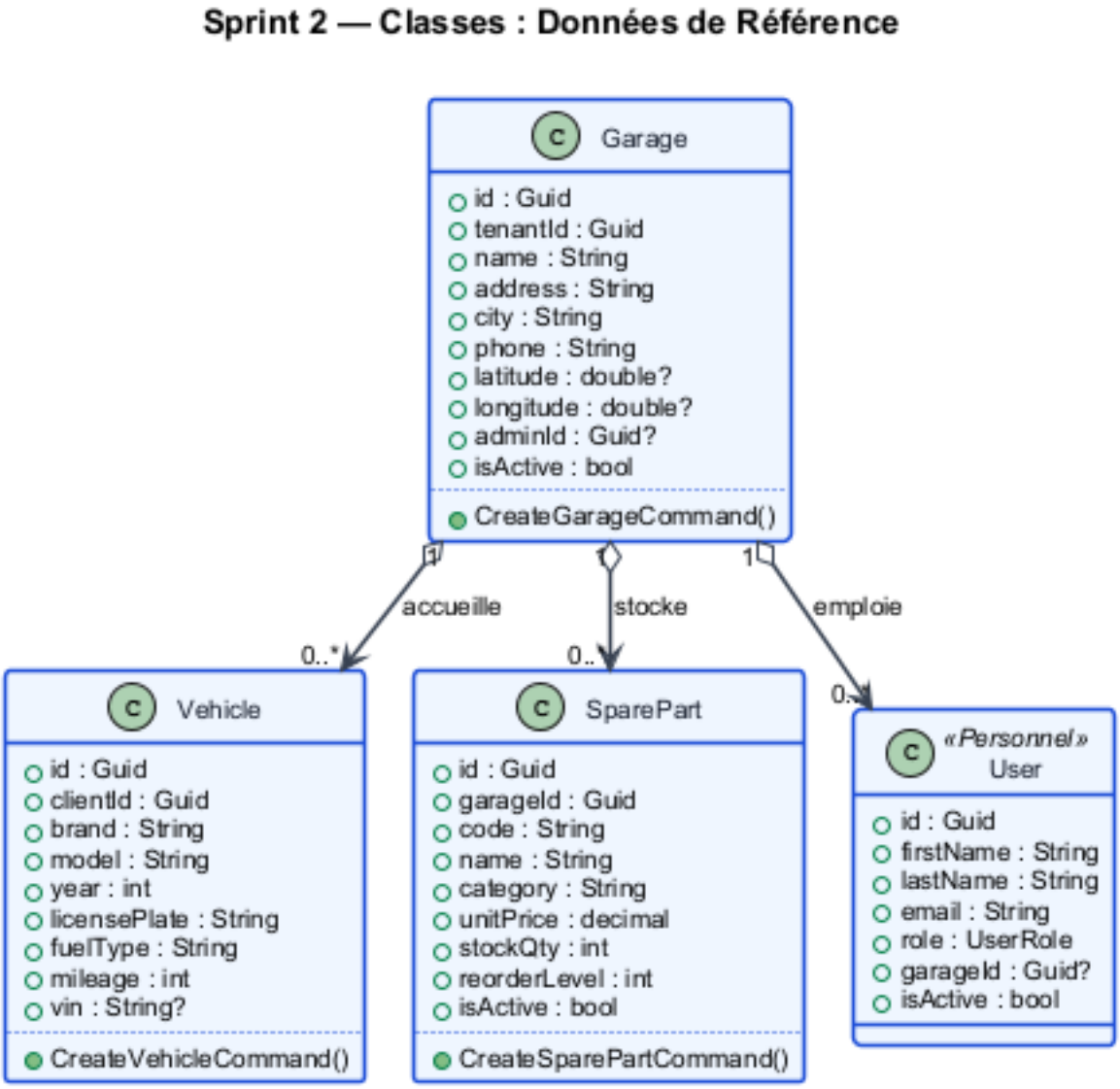


Figure 4.2. Diagramme de classes — Sprint 2 (Données de Référence)

4.3.3 Diagramme de séquence

SC3 — Création d'un Garage (US06) : fonctionnalité principale

L'admin entreprise crée un nouveau garage après vérification de l'existence du tenant.

4.4 Conclusion

Le Sprint 2 a mis en place les données de référence essentielles. Les politiques d'autorisation par rôle ont été appliquées de façon granulaire : seul un `AdminEntreprise` peut créer un garage, et un `Client` ne gère que ses propres véhicules.

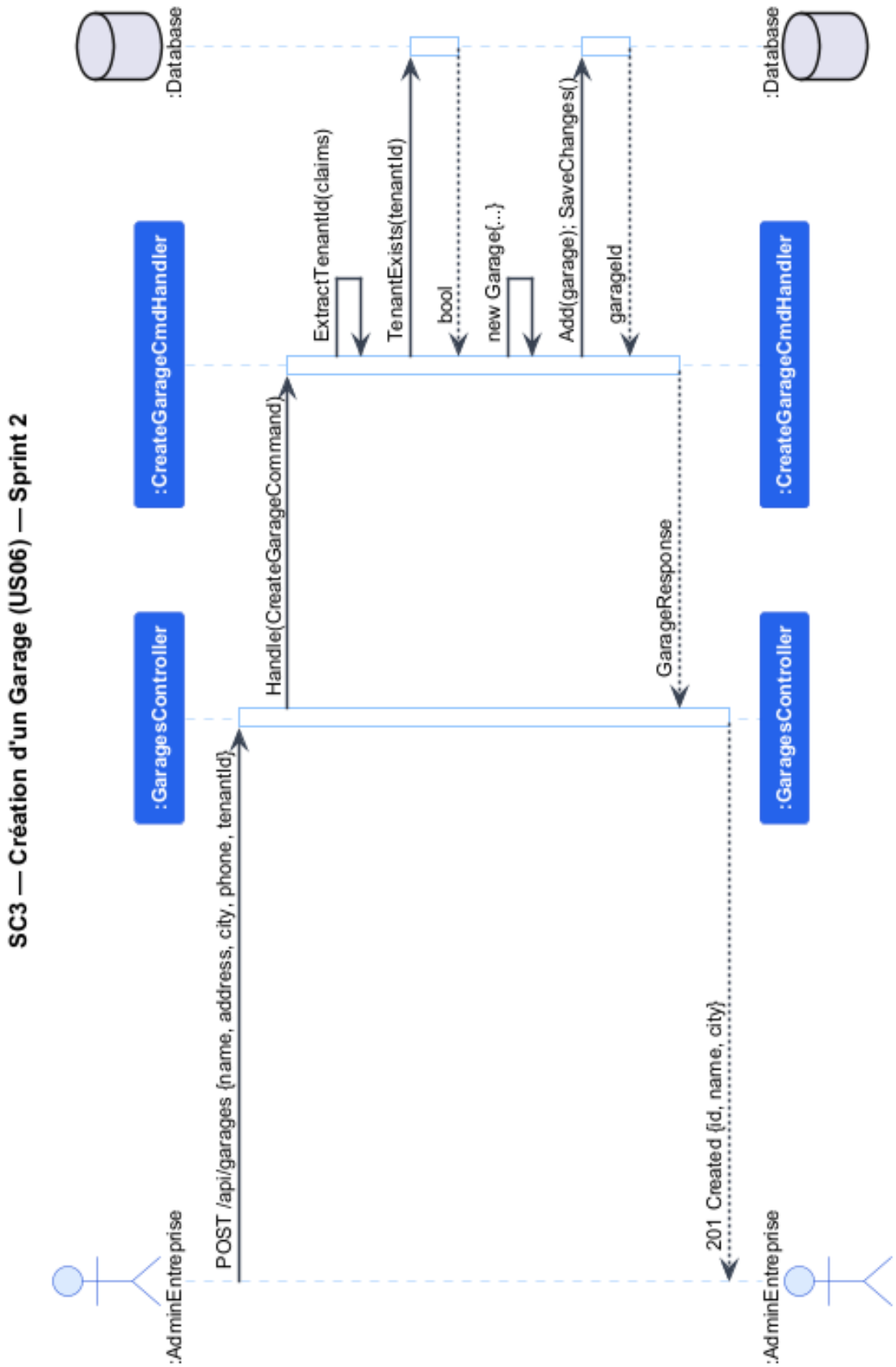


Figure 4.3. Diagramme de séquence SC3 — Création d'un Garage

Chapitre 5

Sprint 3 : Rapports de Symptômes, Diagnostic IA et Rendez-vous

Ce sprint implémente la boucle de diagnostic préliminaire et de prise de rendez-vous. Le client soumet ses symptômes, le service IA génère un diagnostic structuré, le chef d'atelier le révise puis propose une plage, et le client réserve son rendez-vous.

5.1 Introduction

La valeur ajoutée centrale de MecaRage réside dans son module de diagnostic IA. Le service exploite une architecture RAG (Retrieval-Augmented Generation) combinant l'algorithme BM25 pour la recherche dans la base de connaissances mécanique et le modèle **Ollama** (LLM local, open-source) pour la génération du diagnostic structuré.

5.2 Identification du Backlog Sprint 3

Table 5.1. Backlog Sprint 3 — Symptômes, IA et Rendez-vous

ID	User Story	Priorité	SP
US10	Client : soumettre une description textuelle des symptômes	**Haute**	5
US11	Système : déclencher automatiquement le service IA (RAG + Ollama)	**Haute**	8
US12	ChefAtelier : réviser le rapport et proposer une plage de RDV	**Haute**	5
US13	Client : réserver un rendez-vous dans la plage proposée	**Haute**	3
US14	ChefAtelier : approuver ou refuser le rendez-vous	**Haute**	3
		Total Sprint 3	**24 SP**

Tâches techniques :

1. Entités `SymptomReport` et `Appointment`.
2. Service IA FastAPI : architecture RAG BM25 + Ollama (LLM local), endpoint `POST /diagnose`.
3. Commandes : `CreateSymptomReportCommand`, `AddChefFeedbackCommand`, `CreateAppointment` `ApproveAppointmentCommand`.
4. Intégration `HttpClient` côté back-end vers le service IA.
5. Notifications automatiques au chef à chaque nouveau rapport.
6. Interface Angular : formulaire de symptômes, inbox chef, calendrier.

5.3 Conception

5.3.1 Diagramme de cas d'utilisation — Sprint 3

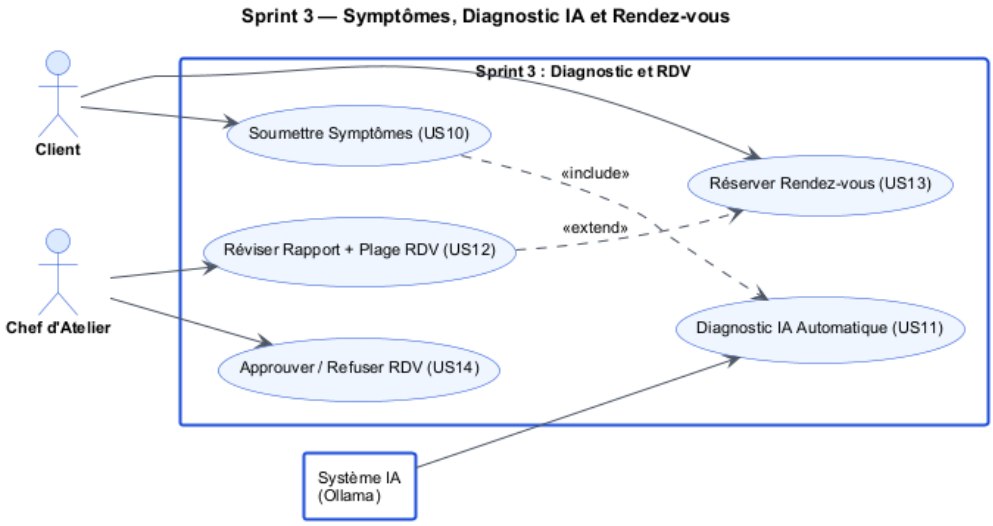
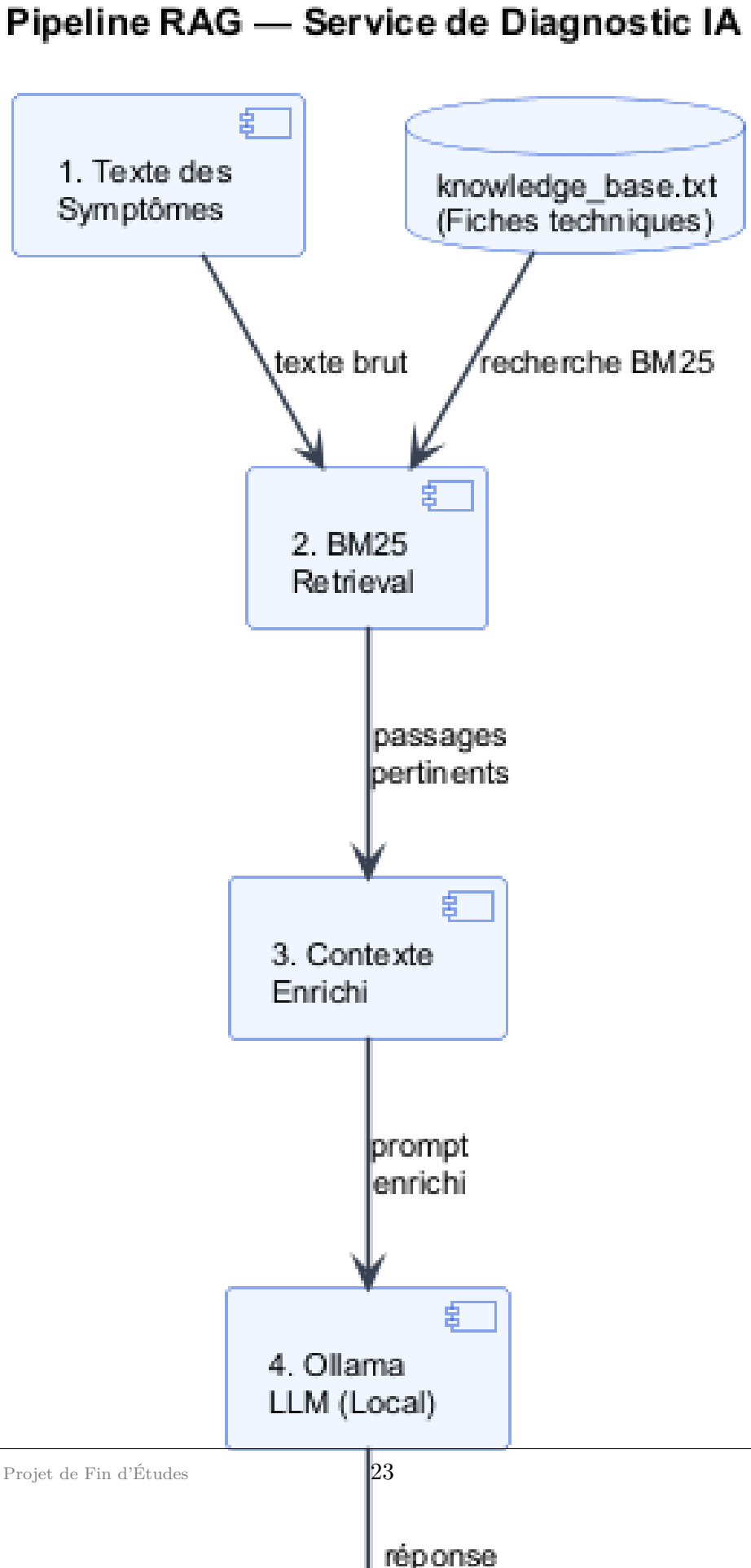


Figure 5.1. Diagramme de cas d'utilisation — Sprint 3

5.3.2 Pipeline du Diagnostic IA



5.3.3 Diagramme de classes — Sprint 3

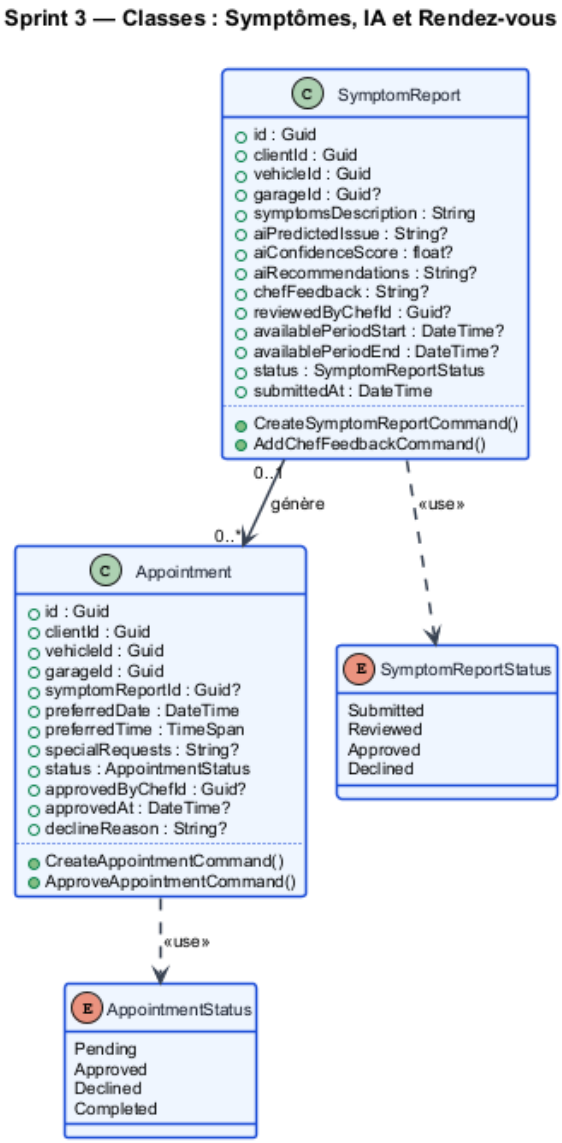


Figure 5.3. Diagramme de classes — Sprint 3 (Symptômes, IA et Rendez-vous)

5.3.4 Diagramme de séquence

SC5 — Soumission de Symptômes et Diagnostic IA (US10+US11) : fonctionnalité principale

Le client soumet des symptômes, le système appelle le service IA, sauvegarde le rapport enrichi, et notifie le chef d'atelier.

5.4 Conclusion

Le Sprint 3 a réalisé la première boucle de valeur visible par le client : soumettre des symptômes, recevoir un diagnostic IA instantané avec score de confiance, obtenir

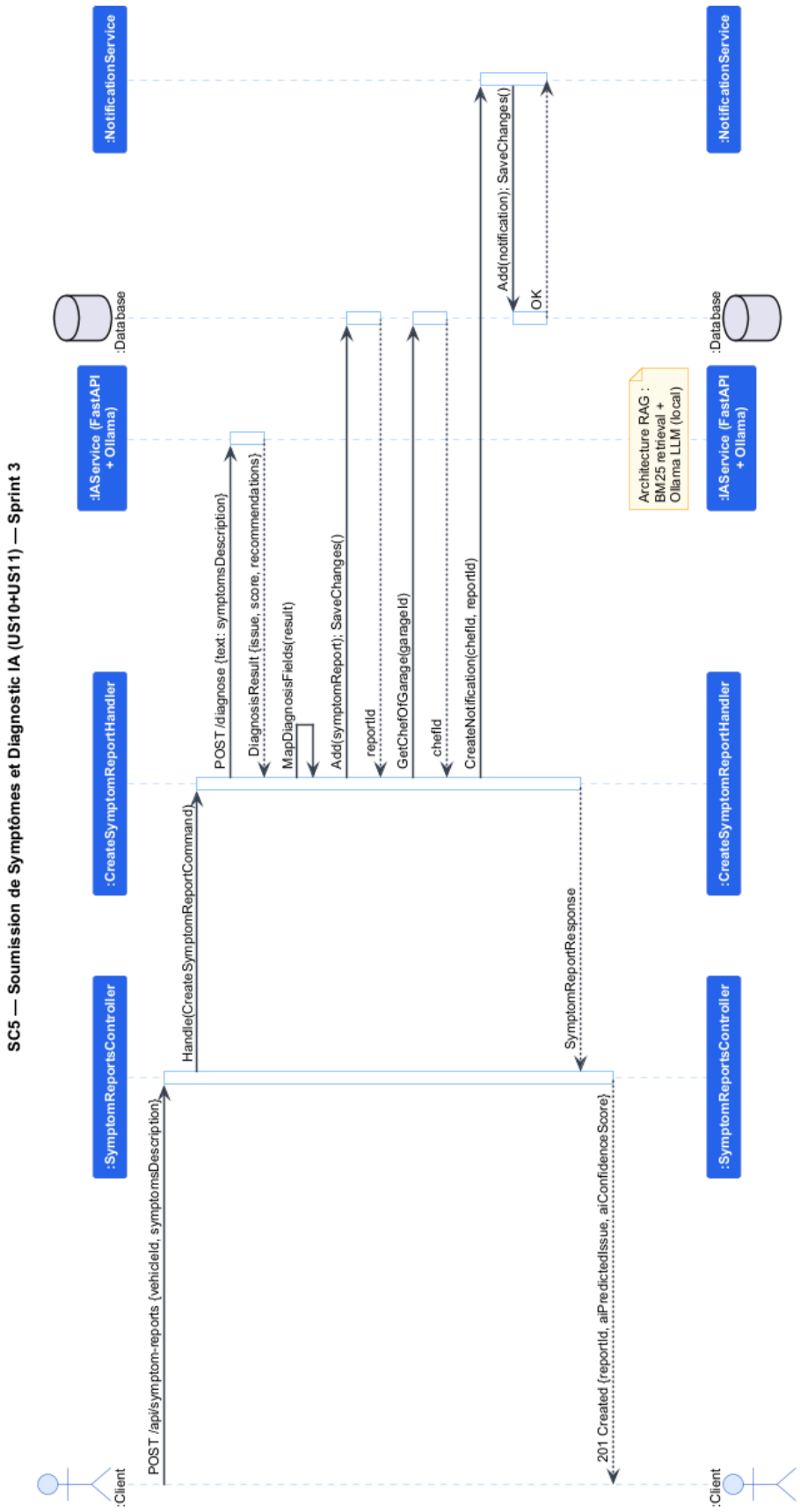


Figure 5.4. Diagramme de séquence SC5 — Soumission de Symptômes et Diagnostic IA

une plage de rendez-vous proposée par le chef, et être notifié de sa confirmation. L'approbation d'un rendez-vous déclenche automatiquement, via MediatR, la création du traceur de cycle de vie de l'intervention.

Chapitre 6

Sprint 4 : Opérations d'Atelier et Cycle de Vie des Interventions

Ce sprint représente le cœur des opérations de l'atelier : affectation des mécaniciens, examens, facturation, approbation client, réparation, validation et paiement. Toutes ces étapes sont tracées automatiquement par le ***Intervention Lifecycle Tracker*** *sur 10 états.*

6.1 Introduction

Une fois le rendez-vous confirmé, le flux opérationnel passe entre les mains de l'équipe de l'atelier. Un tracker transverse se met à jour automatiquement à chaque transition, offrant une visibilité totale sur l'état de chaque intervention.

6.2 Identification du Backlog Sprint 4

Table 6.1. Backlog Sprint 4 — Opérations d'Atelier

ID	User Story	Priorité	SP
US15	Chef : assigner des mécaniciens à une tâche	**Haute**	5
US16	Mécanicien : consulter ses tâches assignées	**Haute**	3
US17	Mécanicien : soumettre un rapport d'examen	**Haute**	5
US18	Chef : réviser et valider le rapport d'examen	**Haute**	3
US19	Chef : créer et finaliser un devis (facture)	**Haute**	5
US20	Client : approuver ou refuser le devis	**Haute**	3
US21	Mécanicien : soumettre la fin de réparation	**Haute**	3
US22	Chef : valider la réparation et notifier le client	**Haute**	3
US23	Admin : enregistrer le paiement	**Haute**	3
US24	Système : tracer le cycle de vie (10 états)	**Haute**	8
US25	Utilisateurs : notifications in-app et push FCM	**Moyenne**	5
US26	Client : télécharger la facture en PDF	**Moyenne**	3
US27	Client / Mécanicien : application Android native	**Moyenne**	13
		Total Sprint 4	**62 SP**

6.3 Conception

6.3.1 Diagramme de cas d'utilisation — Sprint 4

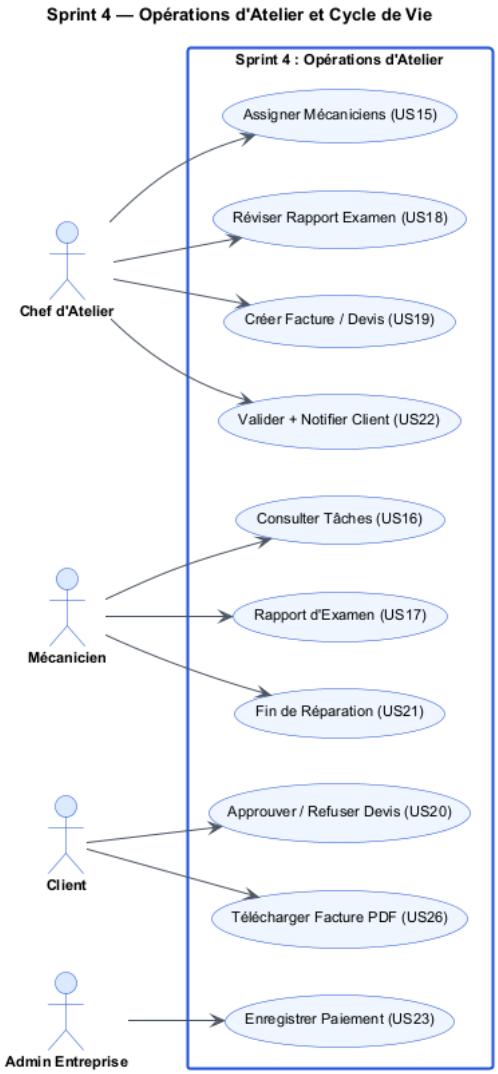
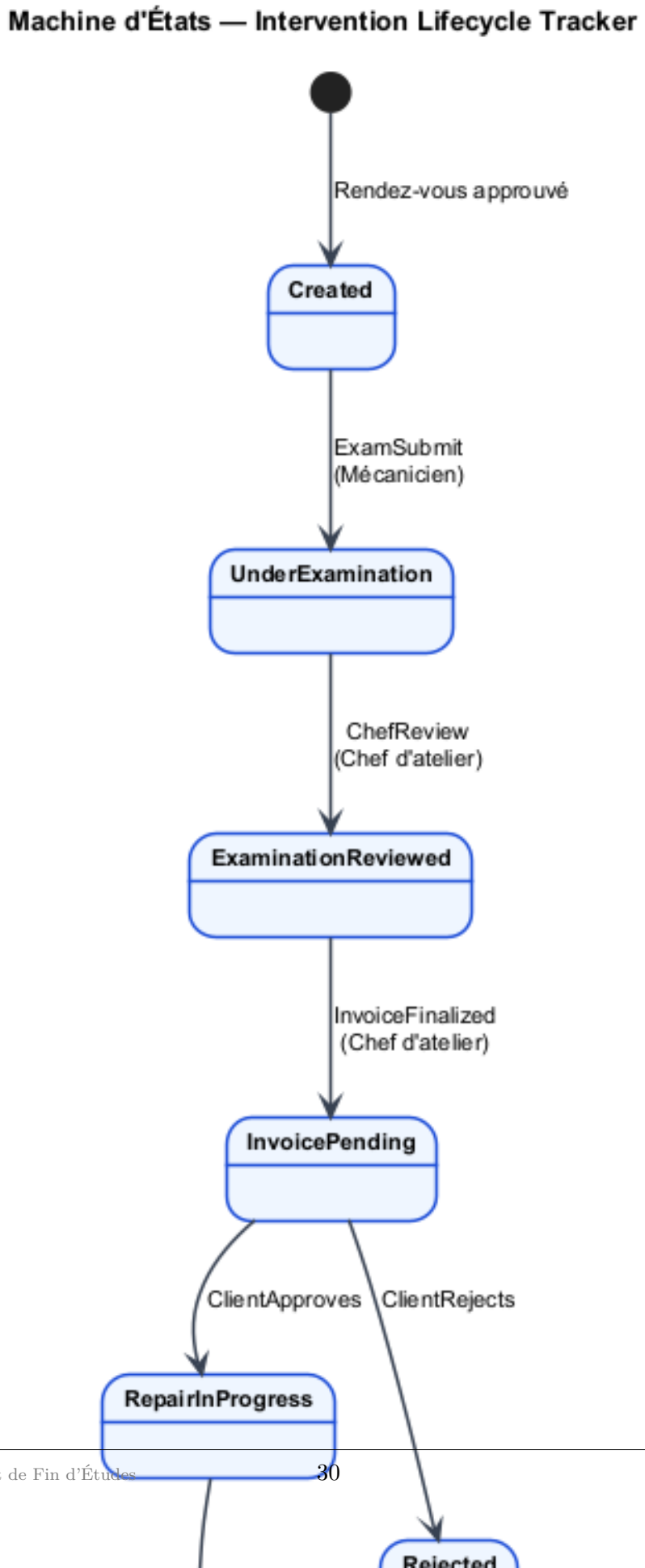


Figure 6.1. Diagramme de cas d'utilisation — Sprint 4

Machine d'états — Intervention Lifecycle Tracker



6.3.2 Diagramme de classes — Sprint 4

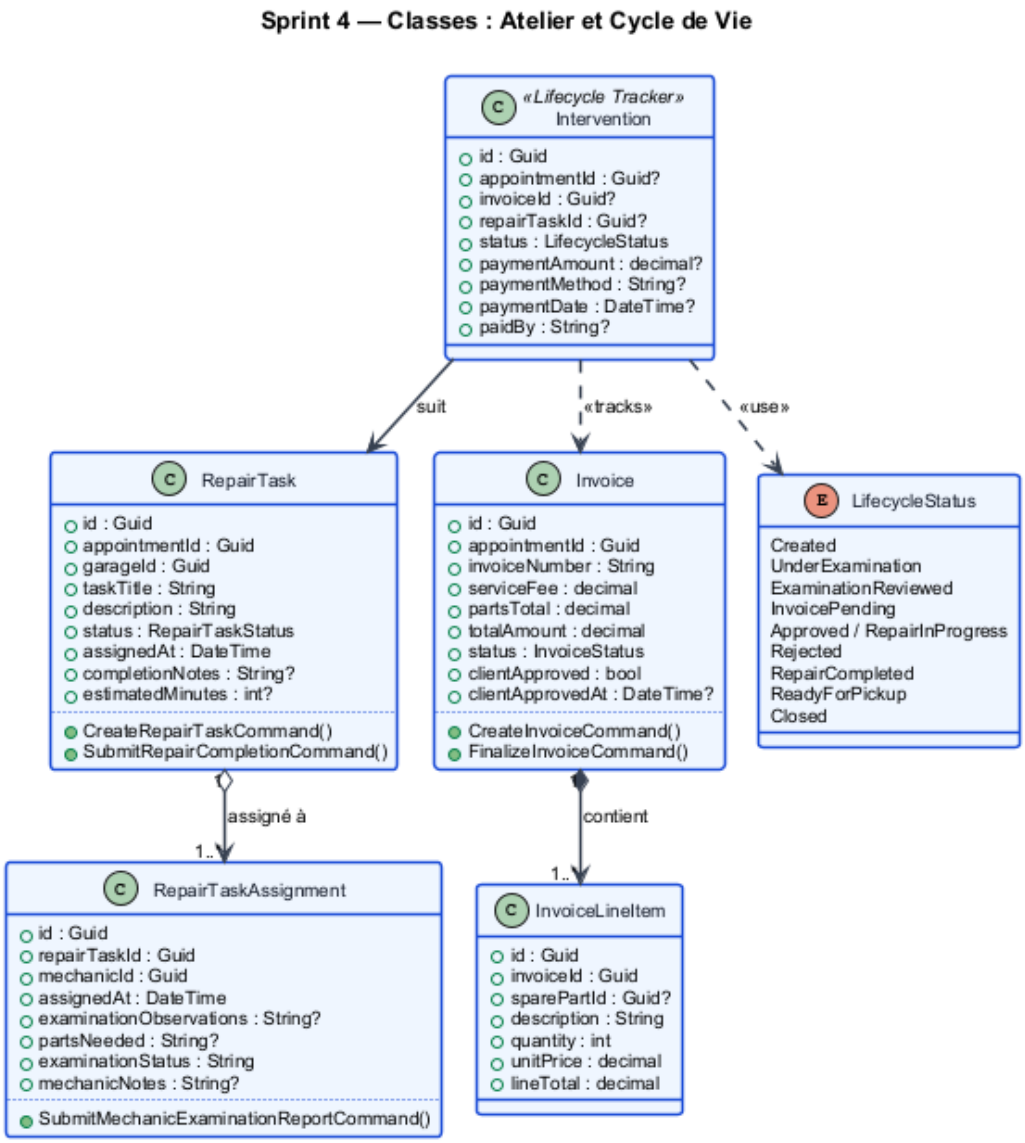


Figure 6.3. Diagramme de classes — Sprint 4 (Atelier et Cycle de Vie)

6.3.3 Diagramme de séquence

SC6 — Finalisation de Facture et Approbation Client (US19+US20) : fonctionnalité principale

Le chef finalise le devis, le client reçoit une notification et l'approuve. Le tracker passe à `InvoicePending` puis `RepairInProgress`.

6.4 Conclusion

Le Sprint 4 a réalisé l'ensemble des opérations métier de l'atelier, de l'affectation des mécaniciens jusqu'à la clôture de l'intervention. Le *Intervention Lifecycle Tracker*, mis

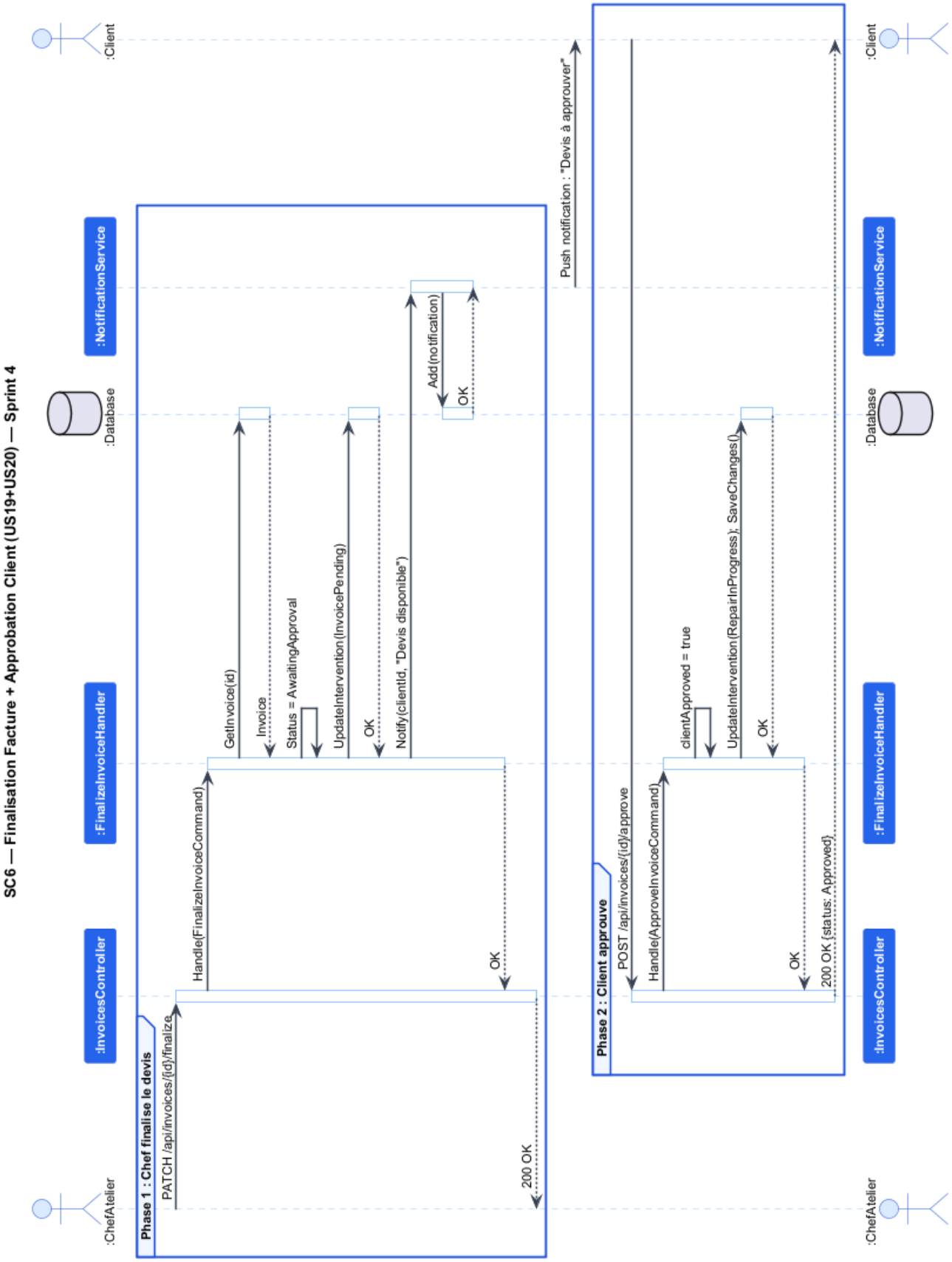


Figure 6.4. Diagramme de séquence SC6 — Finalisation et Approbation du Devis

à jour automatiquement à chaque étape via MediatR, offre une traçabilité complète. La génération de facture PDF (QuestPDF), les notifications FCM (Firebase) et l'application mobile Android complètent l'expérience utilisateur.

Conclusion Générale

Ce chapitre synthétise les réalisations du projet MecaRage et présente les perspectives d'évolution envisagées.

Le projet **MecaRage** représente une solution logicielle complète et cohérente pour la modernisation de la gestion des ateliers mécaniques. À travers quatre sprints successifs, nous avons construit une plateforme qui couvre l'intégralité du cycle de vie d'une intervention automobile : de la réception initiale des symptômes jusqu'au paiement final.

Bilan des réalisations

Table 6.2. Récapitulatif des réalisations par sprint

Sprint	Fonctionnalités	SP réalisés	US
Sprint 1	Authentification JWT, inscription, refresh token, profil	10	4
Sprint 2	Tenants, garages, personnel, véhicules, inventaire pièces	21	5
Sprint 3	Symptômes, diagnostic IA RAG, révision chef, rendez-vous	24	5
Sprint 4	Atelier, facture, approbation, réparation, paiement, PDF, mobile	62	13
	Total	**117 SP**	**27 US**

Les principaux apports techniques du projet sont :

— **Clean Architecture avec CQRS** : séparation des préoccupations, testabilité et maintenabilité à long terme via MediatR.

— **Service de diagnostic IA** : architecture RAG (BM25 + Ollama LLM local) produisant un pré-diagnostic structuré et contextualisé.

— **Intervention Lifecycle Tracker** : suivi transverse automatique sur 10 états, déclenché par les événements existants sans modifier le flux métier.

— **Gestion multi-tenant** : isolation totale des données de chaque entreprise via un filtre global EF Core (`HasQueryFilter`).
— **Application mobile Android (Java + Firebase)** : architecture REST-First avec Firebase comme canal de livraison temps réel (RTDB + FCM).
— **Génération PDF (QuestPDF)** : factures professionnelles téléchargeables directement depuis l'interface client web et mobile.
— **Infrastructure conteneurisée** : Docker Compose supervisé par Prometheus et Grafana pour le monitoring en production.

Perspectives d'évolution

1. **Enrichissement de la base IA** : intégration de nouvelles marques et modèles de véhicules dans `knowledge_base.txt`, fine-tuning du modèle.
2. **Alertes de réapprovisionnement** : décrémenter automatiquement le stock pièces lors de la validation d'examen et alerter sous le seuil de réordre.
3. **Calendrier interactif** : vue mensuelle des rendez-vous via `@fullcalendar/angular` pour une meilleure planification.
4. **Export comptable** : export CSV/Excel des interventions clôturées sur une période pour intégration avec les outils comptables.
5. **Notation et avis client** : évaluation 5 étoiles + commentaire après la clôture d'une intervention.
6. **SignalR** : mise à jour en temps réel du tableau de bord web sans rechargement de page (actuellement polling).
7. **Audit Log** : enregistrement de chaque action sensible via un pipeline MediatR IPipelineBehavior pour la conformité et le débogage.

Bilan personnel

Ce projet nous a permis d'appliquer concrètement des patterns d'architecture logicielle modernes dans un contexte métier réel et contraignant (multi-tenant, multi-rôle, multi-plateforme). L'intégration de technologies hétérogènes — .NET, Angular, Python, Java, Firebase — dans une solution cohérente a constitué le principal défi et la principale source d'apprentissage.

Bibliographie

[1] Microsoft Corporation. *ASP.NET Core 9 Documentation*. Microsoft Docs, 2025. `https://docs.microsoft.com/aspnet/core`

[2] Google LLC. *Firebase Realtime Database Documentation*. Google Developers, 2025. `https://firebase.google.com/docs/database`

[3] Ollama Team. *Ollama — Run Large Language Models Locally*. Ollama, 2025. `https://ollama.com`

[4] Angular Team. *Angular v21 Documentation*. Angular.io, 2025. `https://angular.io/docs`

[5] Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN 0-321-12521-5.

[6] Young, G. *CQRS Documents*. Greg Young, 2010. `https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf`

[7] QuestPDF Team. *QuestPDF — Modern .NET PDF Library*. QuestPDF, 2025. `https://www.questpdf.com`

[8] Robertson, S. & Robertson, J. *Mastering the Requirements Process*. 3e édition. Addison-Wesley, 2012. ISBN 0-321-81574-5.

[9] Pomelo Foundation. *Pomelo.EntityFrameworkCore.MySql — GitHub*. 2025. `https://github.com/PomeloFoundation/Pomelo.EntityFrameworkCore.MySql`

[10] Android Developers. *Android Java — Developer Guide*. Google, 2025. `https://developer.android.com/guide`